

Emotion Detection & Learning Support Engine

Project Description:

The AI-Driven Emotion Detection & Personalized Learning Support Platform is an end-to-end system that transforms a learner's free-text description of their study challenge into an emotion-aware, supportive response. By combining deep learning emotion classifiers (BiLSTM and BERT) with rule-based keyword enhancement and Gemini-generated guidance, the platform ingests raw student input, predicts nuanced emotional states (Bored, Confident, Confused, Curious, Frustrated), and delivers tailored tips, next steps, and encouragement.

Utilizing Google Gemini's API alongside local deep learning classifiers, the system processes user inputs to deliver fast, empathetic learning support. Built with Streamlit for a responsive, dashboard-driven experience, the platform ensures real-time emotional insight and maintains consistent, empathetic responses within seconds. Beyond prediction, the system logs interactions to CSV for continuous learning, visualizes emotion trends through an analytics dashboard, and supports side-by-side model comparisons. This creates a fast, transparent loop that improves learner support quality while minimizing response time and effort.

Scenarios

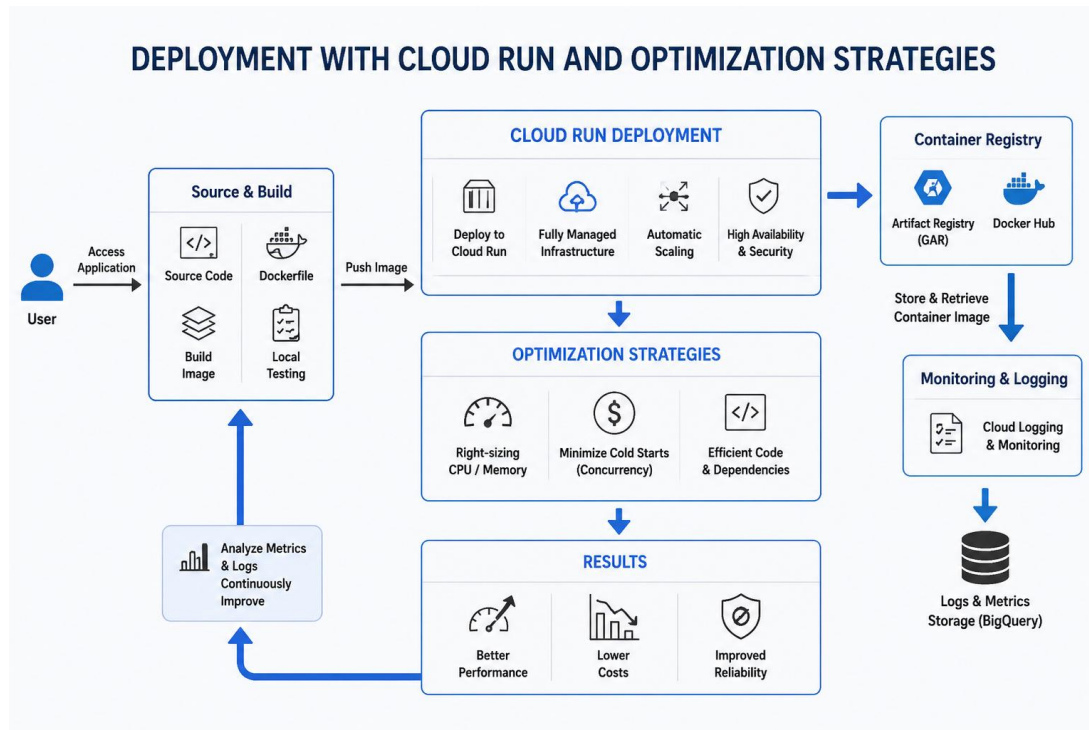
Scenario 1: Educators, TAs & Academic Support Teams

Instructors and academic support staff often spend significant time interpreting student sentiments from messages and emails before offering help. Manual reading, guessing emotions, and crafting the right tone slows response cycles and can miss struggling students. The AI Learning Assistant removes these bottlenecks by automatically classifying emotions from student text, highlighting confidence levels, and generating supportive, field-specific replies. A TA, for instance, can paste a student's concern and instantly see whether they're Confused or Frustrated, along with suggested guidance. Dual-model comparison (BiLSTM vs BERT) and mixed-emotion detection improve reliability and transparency. Teams accelerate triage, maintain consistent empathy, and focus more on targeted academic intervention rather than manual sentiment interpretation.

Scenario 2: Learners & Self-Study

Students working independently often need quick, empathetic guidance when stuck or unsure. Manually searching forums or drafting help requests takes time and may not capture their feelings. The Learning Assistant streamlines this by letting students describe their challenge in plain language and immediately receiving emotion-aware tips and next steps. For example, a learner can note "I'm lost on recursion," and the system will flag Confused, surface clarity-focused advice, and provide encouraging direction. Mixed-emotion outputs (e.g., Curious + Confused) help learners self-reflect, while saved interactions build a history for progress tracking. This makes the platform a powerful companion for self-paced study and rapid clarification.

Technical Architecture:



Description: The platform utilizes a modular multi-tier architecture to deliver rapid, emotion-aware academic feedback. The frontend is built on Streamlit, presenting an interactive user interface for text entry, model configuration, and analytics charts. The backend core coordinates a dual-classifier pipeline: a BiLSTM model and a BERT model process the text to predict five specific emotional states (Bored, Confident, Confused, Curious, Frustrated), incorporating a mixed-emotion mechanism for secondary scores $\geq 15\%$. Once the emotional context is evaluated, the system routes the structured prompt to the Google Gemini API to return tailored study tips and empathetic feedback. Every interaction is continuously appended to a local CSV file to fuel the real-time trend visualization dashboard.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- **Streamlit Framework Knowledge:** [Streamlit Documentation](#)
- **Gemini API Familiarity:** [Google AI Studio Documentation](#)
- **Deep Learning Frameworks:** [TensorFlow/Keras Documentation](#) & [Hugging Face Transformers Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Data Visualizations:** [Plotly / Altair Documentation](#)

Project Workflow:

Epic 1. Environment Setup and Dependency Configuration

- Story 1: Obtain Gemini API Key
- Story 2: Install Python & Create Virtual Environment
- Story 3: Install Project Dependencies
- Story 4: Create the .Env Configuration File
- Story 5: Verify Model and Data Directories
- Story 6: Prepare Project Folder Structure

Epic 2. Kaggle Model Training and Integration

- Story 1: Kaggle Setup (GPU + Dependencies + Data Loading)
- Story 2: Data Preprocessing & Tokenization
- Story 3: BiLSTM Model Training
- Story 4: Domain-Adaptive Fine-Tuning
- Story 5: BERT Model Fine-Tuning
- Story 6: Model Export & Local Integration

Epic 3. Core Emotion Detection Pipeline Development

- Story 1: Text Preprocessing & Keyword Enhancement
- Story 2: BiLSTM Classifier (5-Class Softmax)
- Story 3: BERT Classifier with Class Weighting & Keyword Adjustments
- Story 4: Mixed Emotion Detection ($\geq 15\%$ Secondary Scores)
- Story 5: Unified Prediction Schema
- Story 6: CSV Persistence & Cached Model Loading

Epic 4. AI-Powered Guidance and Regeneration Engine

- Story 1: Capture Field and Problem Context for Prompt Generation
- Story 2: Generate Empathetic and Context-Aware AI Responses
- Story 3: Response Regeneration and Synchronization Mechanism
- Story 4: Session History Management and CSV Logging

Epic 5. Streamlit UI Implementation

- Story 1: Responsive Layout and Session State Management
- Story 2: Emotion Analysis, Model Comparison, and Visualization Components
- Story 3: Form Controls, Validation, and Error Handling
- Story 4: Analytics Dashboard and Interactive Charts

Epic 6. User Interaction

- Story 1: Validate UI Flow End-to-End
- Story 2: Optimization and Deployment Readiness

Epic 1. Environment Setup and Dependency Configuration

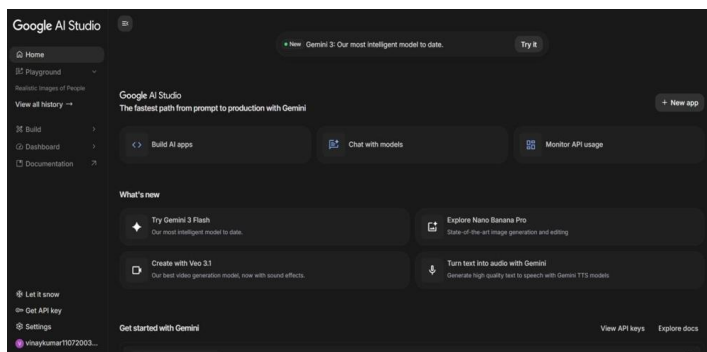
Dependency Configuration) establishes the foundational development environment for the AI Learning Assistant by configuring Python, virtual environments, project dependencies, Gemini API integration, and required model resources. It ensures proper setup of datasets, configuration files, and project structure to support seamless emotion detection, AI-powered guidance, and analytics workflows

Story 1: Obtain Gemini API Key

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

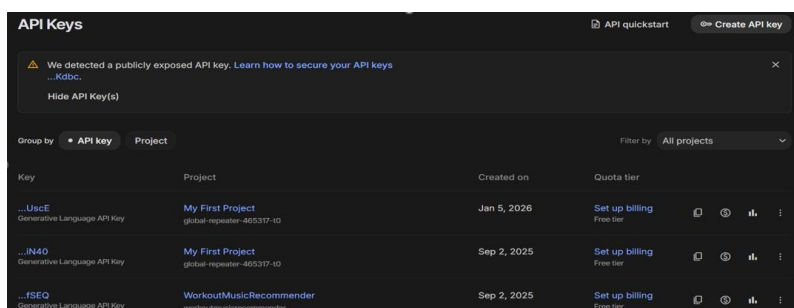
Description

- Visit the official Google AI Studio: <https://aistudio.google.com/>
- Sign in with your Google account to access the Gemini models.



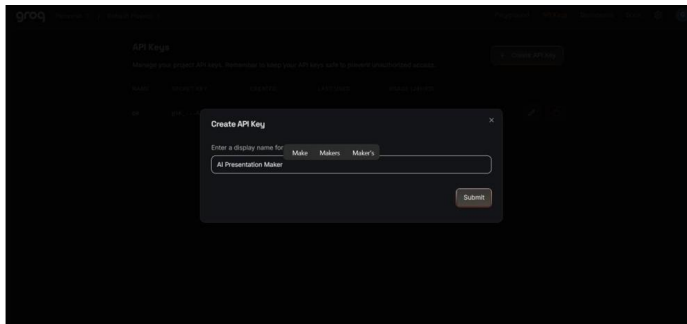
Google AI Studio Home Page

- Click on Get API Key / Create API Key, then generate a key for your project.



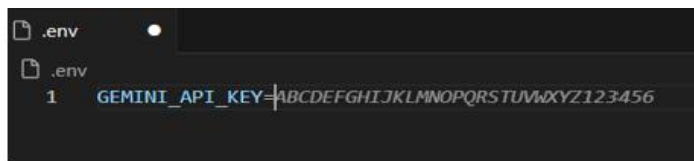
Gemini API Key Creation Screen

- Navigate to the API Keys section in the dashboard.
- Click Create API Key, then assign a recognizable name such as AI_Presentation_Maker.



Creating API Key

- Copy the generated key and store it securely in your project's .env file.

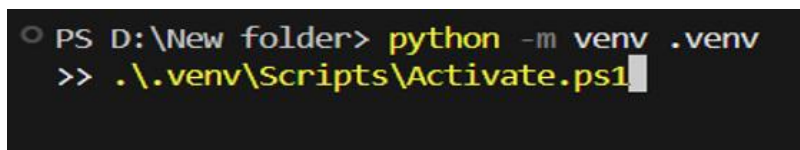


Gemini API Key Saved in. Env

Story 2: Install Python & Create Virtual Environment

Description

- Install **Python 3.9+** from the official website: <https://www.python.org/downloads/>
- Create and activate a virtual environment:



Creating and Activating Python Virtual Environment in Terminal

Story 3: Install Project Dependencies

Description

Use the existing requirements.txt file in the project root to install all required libraries for:

- **Emotion modeling** (TensorFlow/Keras BiLSTM, PyTorch/BERT)
- **NLP** (NLTK)
- **Data handling & analytics** (pandas, numpy, plotly)
- **Web UI & AI integration** (Streamlit, google-generativeai)

```
PS D:\New folder> pip install -r requirements.txt
```

Installing Requirements

Story 4: Create the .Env Configuration File

Description

Add the required environment variables, including:

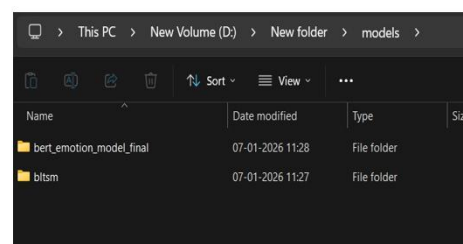
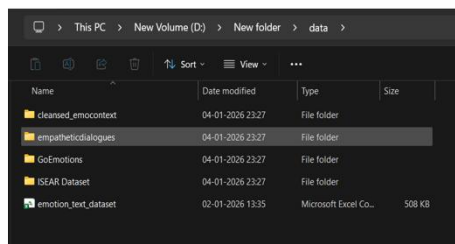
```
.env
1 GEMINI_API_KEY=ABCDEFGHIJKLMNOPQRSTUVWXYZ123456
```

Setting .env File

Story 5: Verify Model and Data Directories

Description

- Ensure that the trained model folders exported from Kaggle are placed correctly:
- **models\bltstm** (BiLSTM model, tokenizer, label classes)
- **models\bert_emotion_model_final** (BERT model, tokenizer, label mappings)
- Confirm that dataset folders under data\ (e.g., emotion_text_dataset.csv, GoEmotions, empatheticdialogues) are present.

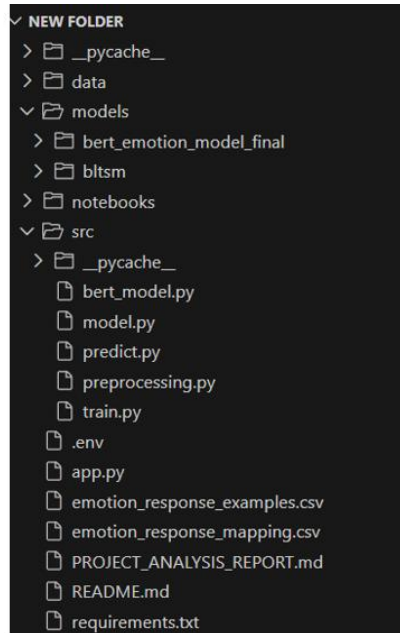


Verification of model folders and its directories

Story 6: Prepare Project Folder Structure

Description

Set Verify that the full project structure is organized as follows:



Folder Structure

Epic 2. Kaggle Model Training and Integration

Story 1: Kaggle Setup (GPU + Dependencies + Data Loading)

Description

The system processes raw student input text through cleaning, tokenization, and keyword enhancement before passing it to the emotion classification models. Each text undergoes specialized preprocessing to preserve emotional context while normalizing format.

- **Environment Configuration:** Utilized Kaggle's dual-GPU environment to accelerate deep learning workflows.
- **Dependency Management:** Installed specific versions of TensorFlow (2.17.0), NumPy (1.26.4), and Hugging Face Transformers to ensure cross-environment stability.
- **Data Verification:** Validated the presence of five core datasets, including **GoEmotions**, **EmpatheticDialogues**, and **ISEAR**, ensuring all CSV paths were correctly mapped.

```
print("Installing Kaggle-safe compatible versions...")
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
               "tensorflow==1.15.0",
               "numpy==1.26.4",
               "pandas==2.1.2",
               "scikit-learn==1.3.2", # FIXED: Works with numpy 1.26.4
               "scipy==1.11.4", # FIXED: Works with numpy 1.26.4
               "matplotlib==3.8.0",
               "seaborn==0.11.0",
               "nltk==3.8.1",
               "transformers==4.35.2", # Simpler, fewer deps
               "torch==1.12",
               "plotly==5.18.0",
               "protobuf==4.24.4"], capture_output=True)

print("Verifying Installation...")
import tensorflow as tf
import numpy as np
import sklearn
import transformers

print(f"TensorFlow: {tf.__version__}")
print(f"NumPy: {np.__version__}")
print(f"scikit-learn: {sklearn.__version__}")
print(f"transformers: {transformers.__version__}")

# GPU setup
gpus = tf.config.list_physical_devices('GPU')
print(f"GPUs detected: {len(gpus)}")
for gpu in gpus:
    tf.config.experimental_set_memory_growth(gpu, True)

print("\nKaggle environment is STABLE - NO CONFLICTS!")
```

```
TensorFlow: 2.15.0
NumPy: 1.26.4
scikit-learn: 1.3.2
transformers: 4.35.2
GPUs detected: 2
Kaggle environment is STABLE - NO CONFLICTS!
```

GPU Detection & Environment Setup

Story 2: Data Preprocessing & Tokenization

Description

LLaMA transforms the extracted content into a structured slide blueprint.

The model generates:

- Unified Dataset Creation: Combined multiple sources into a single dataframe of 198,476 rows with five standardized emotion classes: Bored, Confident, Confused, Curious, and Frustrated.
- Text Cleaning: Implemented regex-based cleaning to normalize text while preserving emotion-carrying punctuation.
- Sequence Preparation: Created a Keras Tokenizer with a 30,000-word vocabulary. Applied padding and truncation to a fixed 80-token length for consistent BiLSTM input.

Text Cleaning & Tokenization

```
# 3. TEXT CLEANING
# =====

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http[s]?://www[.a-z]*", "", text)
    text = re.sub(r"[a-zA-Zs]", "", text)
    tokens = nltk.word_tokenize(text)
    tokens = [t for t in tokens if t not in english_stopwords and len(t) > 1]
    return " ".join(tokens)

print("\nCleaning text...")
combined_df["clean_text"] = combined_df["text"].apply(clean_text)

# 4. TOKENIZATION
# =====

MAX_VOCAB_SIZE = 30000
MAX_SEQ_LEN = 80

tokenizer = Tokenizer(num_words=MAX_VOCAB_SIZE, oov_token="<OOV>")
tokenizer.fit_on_texts(combined_df["clean_text"])

sequences = tokenizer.texts_to_sequences(combined_df["clean_text"])
padded_sequences = pad_sequences(
    sequences, maxlen=MAX_SEQ_LEN, padding="post", truncating="post"
)

print("Tokenization complete:", padded_sequences.shape)
```



```

🔪 Cleaning text...
✅ Tokenization complete: (198476, 80)
🔥 Classes: ['Bored', 'Confident', 'Confused', 'Curious', 'Frustrated']

📁 Saved artifacts:
- padded_sequences.npy
- combined_preprocessed.csv

✅ PREPROCESSING COMPLETE

```

Story 3: BiLSTM Model Training

Description

Slides containing visual content require Stable Diffusion images. Each slide's image prompt is optimized using LLaMA for clarity, realism, and subject relevance.

- **Architecture Design:** Developed a BiLSTM network with an Embedding layer (128-dim) and a Bidirectional LSTM layer (128 units), totaling **4.1 million parameters**.
- **Class Imbalance Handling:** Implemented **Focal Loss** ($\gamma=2.0$) and dataset balancing to ensure the model accurately identifies minority classes like **Bored** and **Frustrated**.
- **Performance Metrics:** Achieved a **95% overall accuracy**, with particularly high F1-scores for **Curious** (1.00) and **Confused** (0.96).

```

# -----
# 4. MODEL (NO cudnn MASK ISSUE)
# -----
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBED_DIM,
        mask_zero=True
    ),
    tf.keras.layers.Bidirectional(
        tf.keras.layers.LSTM(
            LSTM_UNITS,
            dropout=0.2,
            use_cudnn=False # 🐛 FIX mask crash
        )
    ),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(NUM_CLASSES, activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3, clipnorm=1.0),
    loss=focal_loss,
    metrics=["accuracy"]
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)

Model Architecture

```
# -----
# 5. TRAIN
# -----
callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(patience=2, factor=0.5)
]

print("\n🔥 Training (balanced + focal)...")
start = time.time()

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=10,
    batch_size=512,
    callbacks=callbacks,
    verbose=1
)

print(f"\n⌚ Training time: {(time.time()-start)/60:.2f} min")
```

- Shows training progress with focal loss implementation, achieving high accuracy (93.97%) and low loss (0.0643) on training data, with validation accuracy of 83.39%. Training completes in 2.42 minutes with early stopping to prevent overfitting.

```
🔥 Training (balanced + focal)...
Epoch 1/10 23s 262ms/step - accuracy: 0.5859 - loss: 0.6193 - val_accuracy: 0.8459 - val_loss: 0.2444 - learning_rate: 0.0010
Epoch 2/10 18s 257ms/step - accuracy: 0.8411 - loss: 0.2144 - val_accuracy: 0.8153 - val_loss: 0.2319 - learning_rate: 0.0010
Epoch 3/10 18s 254ms/step - accuracy: 0.8900 - loss: 0.1376 - val_accuracy: 0.8243 - val_loss: 0.2271 - learning_rate: 0.0010
Epoch 4/10 17s 252ms/step - accuracy: 0.9107 - loss: 0.1068 - val_accuracy: 0.8076 - val_loss: 0.2569 - learning_rate: 0.0010
Epoch 5/10 18s 255ms/step - accuracy: 0.9193 - loss: 0.0920 - val_accuracy: 0.8411 - val_loss: 0.2221 - learning_rate: 0.0010
Epoch 6/10 17s 252ms/step - accuracy: 0.9276 - loss: 0.0804 - val_accuracy: 0.8267 - val_loss: 0.2448 - learning_rate: 0.0010
Epoch 7/10 17s 252ms/step - accuracy: 0.9295 - loss: 0.0765 - val_accuracy: 0.8094 - val_loss: 0.2584 - learning_rate: 0.0010
Epoch 8/10 17s 247ms/step - accuracy: 0.9397 - loss: 0.0643 - val_accuracy: 0.8339 - val_loss: 0.2527 - learning_rate: 5.0000e-04
⌚ Training time: 2.42 min
```

Training Progress & Focal Loss

- Demonstrates model performance metrics with precision, recall, and f1-score for all 5 emotion classes. Shows high accuracy (95%) overall, with perfect performance on Curious class (1.00 precision/recall) and strong performance on Confused class (0.99 precision, 0.93 recall).

```
# -----
# 6. EVALUATION
# -----
y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)

print("\n📊 CLASSIFICATION REPORT")
print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_,
    digits=4
))

print("\nPrediction distribution:")
print(pd.Series(y_pred).value_counts(normalize=True))
```

```
📊 CLASSIFICATION REPORT
              precision    recall  f1-score   support

    Bored      0.5287      0.6273      0.5738         220
    ...
    0      0.121963
    4      0.117757
```

Classification Model & Model Evaluation

Story 4: Domain-Adaptive Fine-Tuning

Description

Using **python-pptx**, the system builds a polished PowerPoint file using predefined design templates.

- **Targeted Learning:** Fine-tuned the baseline BiLSTM model on 10,000 student-specific samples to better capture academic-related emotional expressions.
- **Transfer Learning Success:** The model achieved 100% validation accuracy on the student dataset, successfully adapting to the specific vocabulary used by learners.
- **Artifact Generation:** Exported the final adaptive model as `bilstm_student_adaptive.keras`.

```
# =====
# 5. CLONE MODEL FOR ADAPTATION
# =====
adaptive_model = tf.keras.models.clone_model(baseline_model)
adaptive_model.set_weights(baseline_model.get_weights())

# Freeze embedding layer
adaptive_model.layers[0].trainable = False

adaptive_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\n Adaptive model ready")
```

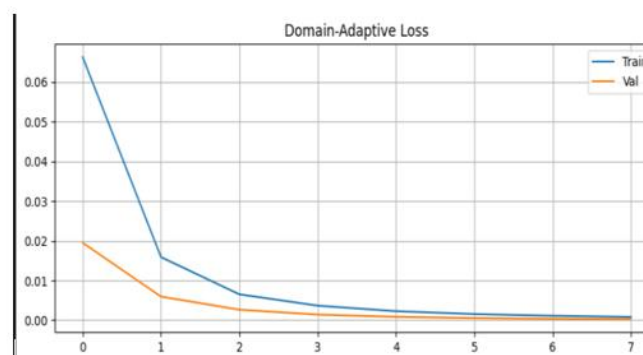
```
# =====
# 6. FINE-TUNE
# =====
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,
        restore_best_weights=True
    )
]

print("\n Fine-tuning on student domain...")

history = adaptive_model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=8,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)
```

Fine-tunes the baseline BiLSTM model on student-specific data

```
 Adaptive model ready
 Fine-tuning on student domain...
Epoch 1/8
125/125 ----- 4s 11ms/step - accuracy: 1.0000 - loss: 0.0663 - val_accuracy: 1.0000 - val_loss: 0.0195
Epoch 2/8
125/125 ----- 1s 8ms/step - accuracy: 1.0000 - loss: 0.0159 - val_accuracy: 1.0000 - val_loss: 0.0059
Epoch 3/8
125/125 ----- 1s 8ms/step - accuracy: 1.0000 - loss: 0.0065 - val_accuracy: 1.0000 - val_loss: 0.0027
Epoch 4/8
125/125 ----- 1s 8ms/step - accuracy: 1.0000 - loss: 0.0037 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 5/8
125/125 ----- 1s 8ms/step - accuracy: 1.0000 - loss: 0.0023 - val_accuracy: 1.0000 - val_loss: 8.5247e-04
Epoch 6/8
125/125 ----- 1s 8ms/step - accuracy: 1.0000 - loss: 0.0016 - val_accuracy: 1.0000 - val_loss: 5.5548e-04
Epoch 7/8
...
macro avg 1.0000 1.0000 1.0000 2000
weighted avg 1.0000 1.0000 1.0000 2000
```



Domain Adaptive Results

Story 5: BERT Model Fine-Tuning

Description

- **Transformer Training:** Fine-tuned bert-base-uncased for three epochs using the AdamW optimizer with a learning rate of 2×10^{-5} .
- **Evaluation:** BERT demonstrated superior contextual understanding, maintaining a 95% accuracy on the test set while showing stable loss reduction ($0.15 \rightarrow 0.05$).
- **Comprehensive Saving:** Exported the full Hugging Face suite, including safetensors, tokenizer.json, and config.json.

```
# Training Loop
from torch.optim import AdamW
from tqdm.auto import tqdm
import numpy as np
from sklearn.metrics import accuracy_score, f1_score

optimizer = AdamW(model.parameters(), lr=2e-5)
num_epochs = 3

for epoch in range(num_epochs):
    # ---- TRAIN ----
    model.train()
    train_losses = []
    for batch in tqdm[Any](train_loader, desc=f"Epoch {epoch+1} - train"):
        batch = {k: v.to(device) for k, v in batch.items()}

        outputs = model(**batch)
        loss = outputs.loss

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        train_losses.append(loss.item())
```

```
*** Epoch 1 - train:  0%|          | 0/5230 [00:00<?, ?it/s]
*** You're using a BertTokenizerFast tokenizer. Please note that with a fast tokenizer, us
*** Epoch 1 - val:  0%|          | 0/454 [00:00<?, ?it/s]
*** Epoch 1: train_loss=0.1558, val_loss=0.1013, val_acc=0.9652, val_f1_macro=0.7303
*** Epoch 2 - train:  0%|          | 0/5230 [00:00<?, ?it/s]
*** Epoch 2 - val:  0%|          | 0/454 [00:00<?, ?it/s]
*** Epoch 2: train_loss=0.0836, val_loss=0.0903, val_acc=0.9697, val_f1_macro=0.7532
*** Epoch 3 - train:  0%|          | 0/5230 [00:00<?, ?it/s]
*** Epoch 3 - val:  0%|          | 0/454 [00:00<?, ?it/s]
*** Epoch 3: train_loss=0.0578, val_loss=0.0981, val_acc=0.9679, val_f1_macro=0.7356
```

BERT Training Loop & Progress

- Shows BERT training progress with decreasing loss ($0.1558 \rightarrow 0.0578$) and improving validation accuracy ($96.52\% \rightarrow 96.79\%$) across 3 epochs. Demonstrates stable training

with consistent f1 - macro scores around 0.73-0.75, indicating good class balance performance.

```
# Test evaluation (from previous message)
test_dataset = HFDataset(tokenized_datasets["test"])
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, collate_fn=data_collator)

model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for batch in tqdm[Any](test_loader, desc="Test"):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        preds = torch.argmax(outputs.logits, dim=-1).cpu().numpy()
        labels = batch["labels"].cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(labels)

from sklearn.metrics import classification_report
print(classification_report(all_labels, all_preds, target_names=list[Any](id2label.values())))
```

BERT Classification Report & Model Saving

- Evaluates fine-tuned BERT model on test set achieving 95% overall accuracy. Shows strong performance on Curious (1.00 precision/recall) and Confused (0.99 precision, 0.93 recall) classes

Story 6: Model Export & Local Integration

Description

- **Convert Directory Structuring:** Migrated cloud-trained artifacts to a local models/ directory, organized into subfolders for bltstm/ and bert_emotion_model_final/.
- **Dependency Matching:** Verified that local versions of the tokenizer and label mappings (metadata) matched the Kaggle training environment exactly.
- **Deployment Readiness:** Confirmed all seven BERT components and three BiLSTM components were correctly placed for Streamlit application loading.

Epic 3. Core Emotion Detection Pipeline Development

Story 1: Text Preprocessing & Keyword Enhancement

Description

This milestone implements the emotion detection pipeline in local code. Processes raw student input through preprocessing, runs BiLSTM and BERT models, detects mixed emotions, and persists results for analytics.

Activity 3.1: Text Preprocessing & Keyword Enhancement

Users can provide instructions:

- Capture Processes raw student input text through cleaning, tokenization, and keyword-based emotion enhancement & Preserves emotional context while normalizing format for model input.

```
def clean_text(self, text):
    text = str(text).lower()
    # Keep punctuation that indicates emotion
    text = re.sub(r'^a-zA-Z\s,!]', ' ', text)
    tokens = nltk.word_tokenize(text)

    # Keep ALL meaningful words, remove only basic articles
    skip_words = {'the', 'a', 'an'}
    tokens = [t for t in tokens if t not in skip_words and len(t) > 1]

    return ' '.join(tokens) if tokens else text
```

Text Cleaning Function

- Keyword scoring with 10x weight for explicit emotions (lines 133-144)
- Probability boosting and renormalization (lines 149-161)

```
125 # Define emotion keywords with higher priority for explicit emotions
126 emotion_keywords = {
127     'Frustrated': ['frustrated', 'frustrating', 'annoying', 'angry', 'hate', 'difficult', 'stuck', 'wrong answer', 'keep getting', 'unnecessarily complicated', 'bribe',
128     'Curious': ['why', 'how', 'what', 'curious', 'wonder', 'interested', 'learn', 'know more', 'want to know', 'explore', 'could we', 'what happens', 'intuition', 'huh',
129     'Confident': ['easy', 'amazing', 'great', 'excellent', 'good', 'awesome', 'perfect', 'solved', 'got it', 'clear now', 'finally', 'move ahead', 'understand clearly',
130     'Bored': ['boring', 'bored', 'tired', 'repetitive', 'dull', 'not engaging', 'didn't feel engaging', 'didn't feel engaging', 'not interesting', 'too basic', 'losing',
131     'Confused': ['confused', 'lost', 'unclear', 'don't understand', 'don't understand', 'doesn't make sense', 'not fully confident', 'missing', 'incomplete', 'unsure',
132 }
133
134 # Score each emotion based on keyword matches with higher weights for explicit emotions
135 emotion_scores = {}
136 for emotion, keywords in emotion_keywords.items():
137     score = 0
138     for keyword in keywords:
139         if keyword in text_lower:
140             # Give much higher weight to explicit emotion words
141             if keyword in ['frustrated', 'frustrating', 'curious', 'confident', 'bored', 'boring', 'confused']:
142                 score += 10 # Very high weight for explicit emotions
143             else:
144                 score += 2
145     emotion_scores[emotion] = score
146
147 # Pick to add a breakpoint for emotion with highest keyword score
148 max_score = max(emotion_scores.values())
149 if max_score > 0:
150     # Boost the emotion(s) with highest keyword matches
151     for emotion, score in emotion_scores.items():
152         if score == max_score:
153             emotion_idx = self.classes.index(emotion)
154             probs[emotion_idx] *= (1 + score * 3.0) # Very strong boost for keyword matches
155
156 # Reduce other emotions more aggressively
157 winning_emotions = [e for e, s in emotion_scores.items() if s == max_score]
158 for i, emotion in enumerate(self.classes):
159     if emotion not in winning_emotions and max_score > 5: # Lower threshold for strong override
160         probs[i] *= 0.01 # Very strong reduction
161
162 probs = probs / np.sum(probs) # Renormalize
```

Keyword Enhancement Logic

Story 2: BiLSTM Classifier (5-Class Softmax)

Description

- Loads the trained BiLSTM model and generates emotion probability distributions using softmax activation.
- Processes tokenized sequences through the neural network to predict 5 emotion classes.

```
class EmotionPredictor:
    def __init__(self, model_path="models/blstm/blstm_student_adaptive.keras", tokenizer_path="models/blstm/tokenizer.json", classes_path="models/blstm/label_classes.json"):
        try:
            self.model = tf.keras.models.load_model(model_path)
        except:
            # Try loading without compiling for models with custom loss functions
            self.model = tf.keras.models.load_model(model_path, compile=False)

        with open(tokenizer_path, 'r') as f:
            self.tokenizer = tokenizer_from_json(f.read())

        with open(classes_path, 'r') as f:
            self.classes = json.load(f)

        self.stopwords = set([any](stopwords.words('english')))
```

BiLSTM Model Loading

- Explanation: Converts cleaned text to token sequences, pads to fixed length (80 tokens), and generates emotion probabilities through BiLSTM model inference.
- Applies softmax normalization to ensure probabilities sum to 1.0 for all 5 emotion classes.

```
def predict(self, text: str) -> Dict[str, Any]:
    cleaned = self.clean_text(text)

    if not cleaned.strip():
        cleaned = text.lower()

    sequence = self.tokenizer.texts_to_sequences([cleaned])

    if not sequence or not sequence[0]:
        return {
            'emotion': 'Confused',
            'confidence': 0.5,
            'scores': {cls: 0.2 for cls in self.classes},
            'cleaned_text': cleaned
        }

    padded = pad_sequences(sequence, maxlen=MAX_SEQ_LEN, padding='post')

    probs = self.model.predict(padded, verbose=0)
    probs = np.array(probs).flatten()

    if len(probs) != len(self.classes):
        probs = np.resize(probs, len(self.classes))

    # Apply softmax for better probability distribution
    probs = np.exp(probs) / np.sum(np.exp(probs))
```

BiLSTM Prediction & Softmax

Story 3: BERT Classifier with Class Weighting & Keyword Adjustments

Description

- Loads the fine-tuned BERT model and applies class weighting and keyword-based adjustments for confidence/confusion detection.
- Generates transformer-based emotion predictions with enhanced accuracy.

```

bert_model.py 4 X
src > bert_model.py > BERTEmotionClassifier
9 class BERTEmotionClassifier:
18
19 def load_model(self, model_path='models/bert_emotion_model_final'):
20     self.tokenizer = AutoTokenizer.from_pretrained(model_path)
21     self.model = AutoModelForSequenceClassification.from_pretrained(model_path)
22     self.model.to(self.device)
23
24     # Load label mappings if available
25     label_path = os.path.join(model_path, 'label_mappings.pkl')
26     if os.path.exists(label_path):
27         with open(label_path, 'rb') as f:
28             mappings = pickle.load(f)
29             if 'id2label' in mappings:
30                 self.id2label = mappings['id2label']
31                 self.emotion_labels = [mappings['id2label'][i] for i in range(len(mappings['id2label']))]

```

BERT Model Loading

Tokenizes input text, generates BERT predictions, and applies class weighting (1.2, 1.8, 0.6, 1.0, 1.4 for Bored, Confident, Confused, Curious, Frustrated).

Enhances predictions with keyword-based adjustments: boosts Confident class (2.5x) when confidence keywords are detected or boosts Confused class (2.0x) when confusion keywords are found

```

bert_model.py 4 X
src > bert_model.py > BERTEmotionClassifier
9 class BERTEmotionClassifier:
32
33 def predict(self, text):
34     if self.model is None:
35         raise ValueError("Model not loaded. Call load_model() first.")
36
37     inputs = self.tokenizer(text, return_tensors="pt", truncation=True, padding=True)
38     inputs = {k: v.to(self.device) for k, v in inputs.items()}
39
40     with torch.no_grad():
41         outputs = self.model(**inputs)
42         probs = torch.softmax(outputs.logits, dim=-1).cpu().numpy()[0]
43
44     # Enhanced class weights with confidence keywords boost
45     class_weights = np.array([1.2, 1.8, 0.6, 1.0, 1.4]) # Bored, Confident, Confused, Curious, Frustrated
46
47     # Keyword-based adjustments
48     text_lower = text.lower()
49     confidence_keywords = ['comfortable', 'confident', 'easy', 'clear', 'understand', 'got it', 'makes sense']
50     confusion_keywords = ['confused', 'unclear', 'lost', 'don't understand', 'puzzled']
51
52     # Boost confidence if confidence keywords found
53     if any(keyword in text_lower for keyword in confidence_keywords):
54         class_weights[1] *= 2.5 # Boost Confident
55         class_weights[2] *= 0.3 # Reduce Confused
56
57     # Only boost confusion if explicit confusion keywords found
58     elif any(keyword in text_lower for keyword in confusion_keywords):
59         class_weights[2] *= 2.0 # Boost Confused
60
61     weighted_probs = probs * class_weights
62     pred_id = np.argmax(weighted_probs)
63
64     emotion = self.id2label[pred_id]
65
66     return {
67         "emotion": emotion,
68         "confidence": float(weighted_probs[pred_id] / np.sum(weighted_probs)),
69         "scores": {self.id2label[i]: float(weighted_probs[i] / np.sum(weighted_probs)) for i in range(5)},
70         "cleaned_text": text.strip()
71     }

```

BERT Prediction with Class Weighting

Story 4: Mixed Emotion Detection ($\geq 15\%$ Secondary Scores)

Description

- Analyzes emotion scores and identifies emotions above 15% confidence threshold to detect multiple simultaneous emotions.
- Provides nuanced emotional insight beyond single-label classification.

```

app.py 5 X
app.py > ...
184 def main():
300
301     # Function to detect mixed sentiment
302     def get_mixed_emotions(scores, threshold=0.15):
303         sorted_emotions = sorted(scores.items(), key=lambda x: x[1], reverse=True)
304         primary = sorted_emotions[0]
305         mixed = [primary]
306
307         for emotion, score in sorted_emotions[1:]:
308             if score >= threshold:
309                 mixed.append((emotion, score))
310
311         return mixed if len(mixed) > 1 else [primary]
  
```

Mixed-Emotion Detection Function

```

app.py 5 X
app.py > ...
184 def main():
315
316     bilstm_mixed = get_mixed_emotions(bilstm_result['scores'])
317
318     if len(bilstm_mixed) > 1:
319         # Mixed sentiment display
320         mixed_text = " + ".join([f"{EMOTION_RESPONSES[em[0]]['emoji']} {em[0]}" for em in bilstm_mixed])
321         st.metric("Mixed Emotions", mixed_text, f"Primary: {bilstm_mixed[0][1]:.1%}")
322     else:
323         # Single emotion display
324         bilstm_emoji = EMOTION_RESPONSES[bilstm_result['emotion']]['emoji']
325         st.metric("Emotion", f"{bilstm_emoji} {bilstm_result['emotion']}", f"{bilstm_result['confidence']:.1%}")
326
327     for emotion_name, score in sorted(bilstm_result['scores'].items(), key=lambda x: x[1], reverse=True):
328         st.progress(score, text=f"{emotion_name}: {score:.1%}")
329
330     if bert_result:
331         with col2:
332             st.write("***BERT Transformer***")
333             bert_mixed = get_mixed_emotions(bert_result['scores'])
334
335             if len(bert_mixed) > 1:
336                 mixed_text = " + ".join([f"{EMOTION_RESPONSES[em[0]]['emoji']} {em[0]}" for em in bert_mixed])
337                 st.metric("Mixed Emotions", mixed_text, f"Primary: {bert_mixed[0][1]:.1%}")
  
```

Mixed-Emotion Display in UI

- Displays combined emotions (e.g., "Curious + Confused") when multiple emotions are detected above threshold, showing primary emotion confidence.
- Falls back to single emotion display when only one emotion exceeds the threshold.

Story 5: Unified Prediction Schema

Description

- Creates a standardized result format containing emotion, confidence, all emotion scores, and cleaned text.
- Ensures consistent output structure across both BiLSTM and BERT models.

```

113     }
114     ,c769u6q_f6xf,: c769u6q
115     ,zcol62,: zcol62'
116     ,countq6uc6,: countq6uc6'
117     ,emotion,: emotion'
118     return {
119
120         zcol62 = {z6f'.c792262[1]: t709f(bl0p2[1]) }ou.7 fu u9u86(76u(z6f'.c792262))}
121         countq6uc6 = t709f(bl0p2[emotion_ix])
122         emotion = z6f'.c792262[emotion_ix]
123         emotion_ix = ub.9l8u9x(bl0p2)
124
125         q6f bl0p2f(z6f'. f6xf: zfl) -> D1cf[zfl, yuyl]:
126         c7922 f709fou76u7cfou:
127
128     ac > buebioc62ziub'bl > ""
129     buebioc62ziub'bl e x
  
```

BiLSTM Prediction Schema

- Generates unified prediction schema matching BiLSTM format, with emotion, confidence (normalized weighted probability), all emotion scores dictionary, and cleaned text.
- Ensure consistent output structure for both models.

```

bert_model.py 4 X
src > bert_model.py > BERTEmotionClassifier
9 class BERTEmotionClassifier:
33     def predict(self, text):
65
66         return {
67             "emotion": emotion,
68             "confidence": float(weighted_probs[pred_id] / np.sum(weighted_probs)),
69             "scores": {self.id2label[i]: float(weighted_probs[i] / np.sum(weighted_probs)) for i in range(5)},
70             "cleaned_text": text.strip()
71         }
  
```

BERT Prediction Schema

Story 6: CSV Persistence & Cached Model Loading

Description

- Saves all interactions to CSV files for continuous learning and analytics.

- Implements Streamlit caching to load models once and reuse them across sessions for improved performance.

```

app.py 5 X
app.py > ...
24
25 @st.cache_resource
26 def load_models():
27     try:
28         bilstm_model = EmotionPredictor("models/blstm/bilstm_student_adaptive.keras")
29
30         # Load BERT model if available
31         bert_model = None
32         try:
33             bert_model = BERTEmotionClassifier()
34             bert_model.load_model('models/bert_emotion_model_final')
35         except:
36             pass
37
38         return bilstm_model, bert_model, "✅ Models loaded"
39     except Exception as e:
40         return None, None, f"❌ Error: {e}"
  
```

Cached Model Loading

- Saves each interaction to emotion_response_examples.csv with text, emotion, confidence, response, field, and timestamp.
- Updates emotion_response_mapping.csv with new emotion-response pairs for future reference. Enables continuous learning and analytics by persisting all user interactions

```

app.py 5 X
app.py > ...
104
105 def save_to_csv(field, problem, emotion, confidence, ai_response):
106     """Save new interaction to CSV files."""
107     try:
108         # Update examples CSV
109         new_example = {
110             'text': problem,
111             'emotion': emotion.lower(),
112             'confidence': confidence,
113             'response': ai_response,
114             'field': field,
115             'timestamp': datetime.now().isoformat()
116         }
117
118         # Check if file exists
119         if os.path.exists("emotion_response_examples.csv"):
120             df = pd.read_csv("emotion_response_examples.csv")
121             df = pd.concat([df, pd.DataFrame([new_example])], ignore_index=True)
122         else:
123             df = pd.DataFrame([new_example])
124
125         df.to_csv("emotion_response_examples.csv", index=False)
126
127         # Update mapping CSV if new emotion-response pair
128         if os.path.exists("emotion_response_mapping.csv"):
129             mapping_df = pd.read_csv("emotion_response_mapping.csv")
130             if emotion not in mapping_df['emotion'].values:
131                 new_mapping = pd.DataFrame([{'emotion': emotion, 'response': ai_response}])
132                 mapping_df = pd.concat([mapping_df, new_mapping], ignore_index=True)
133                 mapping_df.to_csv("emotion_response_mapping.csv", index=False)
134
135         return True
136     except Exception as e:
137         st.error(f"Failed to save to CSV: {e}")
138         return False
  
```

BERT Prediction Schema

Epic 4. AI-Powered Guidance and Regeneration Engine

Story 1: Capture Field and Problem Context for Prompt Generation

Description

- Captures student's academic field through dropdown selection (11 options) and problem description through text area input.

- Provides contextual placeholders based on selected fields to guide user input

```

app.py 5 X
app.py 7 save_to_csv
184
218
219 # Field selection
220 field = st.selectbox(
221     "What field are you studying?",
222     ["Computer Science", "Mathematics", "Physics", "Chemistry", "Biology",
223      "Engineering", "Business", "Literature", "History", "Psychology", "Other"],
224     help="Select your area of study for personalized responses"
225 )
226
227 # Problem description
228 problem = st.text_area(
229     f"Describe your {field} problem or challenge:",
230     placeholder=f"e.g., 'I'm struggling with algorithms in {field}' or 'This concept is confusing'",
231     height=120
232 )
233

```

Field & Problem Input Capture

- Constructs personalized prompts for Gemini AI that include the student's study field, detected emotion, confidence percentage, and problem description.
- Structures the prompt to request acknowledgment, field-specific tips, and encouraging next steps. Handles API errors gracefully with fallback messages.

```

app.py 5 X
app.py 7 save_to_csv
82
83 return None
84
85 def get_gemini_response(field, problem, emotion, confidence):
86     """Get AI response from Gemini based on field and problem."""
87     try:
88         prompt = f"""
89         You are a helpful learning assistant. A student studying {field} is feeling {emotion} (confidence: {confidence:.1%}) about this problem:
90         "{problem}"
91
92         Provide a clear, supportive response with:
93         1. Brief acknowledgment of their feeling
94         2. One specific tip or strategy for {field}
95         3. One encouraging next step
96
97         Use simple, clear language. Keep each point to 1-2 sentences. No markdown formatting.
98         """
99         response = model_gemini.generate_content(prompt)
100         return response.text.strip()
101     except Exception as e:
102         return f"AI response unavailable: {e}"
103

```

Gemini Prompt Construction

- Integrates detected emotion and confidence score from BiLSTM model into Gemini prompt construction.
- Uses BiLSTM prediction results (emotion and confidence) as input parameters for generating personalized AI responses.

```
app.py 5 X
app.py > save_to_csv
184 def main():
262
263     # Analysis button
264     if st.button("🔍 Get AI Learning Help", type="primary", use_container_width=True):
265         if problem.strip():
266             with st.spinner("Analyzing your learning state..."):
267                 # Get predictions from models
268                 bilstm_result = bilstm_model.predict(problem)
269                 bert_result = bert_model.predict(problem) if bert_model else None
270
271                 # Use BiLSTM result for AI response (or you can choose based on confidence)
272                 emotion_result = bilstm_result
273                 emotion = emotion_result['emotion']
274                 confidence = emotion_result['confidence']
275
```

Emotion & Confidence Integration

Story 2: Generate Empathetic and Context-Aware AI Responses

Description

- Configures Gemini 2.5 Flash model with API key from environment variables and generates content based on constructed prompt. Returns cleaned response text with striped whitespace for display.

```
# Configure Gemini API
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model_gemini = genai.GenerativeModel('gemini-2.5-flash')

# In get_gemini_response function:
response = model_gemini.generate_content(prompt)
return response.text.strip()
```

Gemini API Response Generation

- Provides fallback mechanism to predefined template responses when Gemini API is unavailable, disabled, or encounters errors. Each emotion has a corresponding emoji,

```
app.py X
app.py > ...
184 def main():
262
263     # Analysis button
264     if st.button("🔍 Get AI Learning Help", type="primary", use_container_width=True):
265         if problem.strip():
266             with st.spinner("Analyzing your learning state..."):
267                 # Get predictions from models
268                 bilstm_result = bilstm_model.predict(problem)
269                 bert_result = bert_model.predict(problem) if bert_model else None
270
271                 # Use BiLSTM result for AI response (or you can choose based on confidence)
272                 emotion_result = bilstm_result
273                 emotion = emotion_result['emotion']
274                 confidence = emotion_result['confidence']
275
276                 # Get AI response
277                 if use_ai:
278                     ai_response = get_gemini_response(field, problem, emotion, confidence)
279                 else:
280                     ai_response = EMOTION_RESPONSES[emotion]['response']
281
282                 # Save to CSV
283                 if save_data:
284                     save_success = save_to_csv(field, problem, emotion, confidence, ai_response)
285                     if save_success:
286                         st.success("💾 Interaction saved to improve future responses!")
287
288                 # Add to session history
289                 add_to_history(field, problem, emotion, confidence, ai_response, bilstm_result['scores'], bert_result)
290
```

supportive response, and suggested action for consistent user experience.

```
# Get AI response
if use_ai:
    ai_response = get_gemini_response(field, problem, emotion, confidence)
else:
    ai_response = EMOTION_RESPONSES[emotion]['response']
```

```
preprocessing.py 6 X
src > preprocessing.py > ...

256
257 EMOTION_RESPONSES = {
258     'Confused': {
259         'emoji': '😞',
260         'response': 'I see you might be confused. Let me break this down step-by-step...',
261         'action': 'Show detailed explanation'
262     },
263     'Frustrated': {
264         'emoji': '😡',
265         'response': 'I understand this is frustrating! Let's try a simpler approach...',
266         'action': 'Suggest alternative learning path'
267     },
268     'Confident': {
269         'emoji': '😎',
270         'response': 'Great! You're making excellent progress! Ready for the next challenge?',
271         'action': 'Suggest advanced content'
272     },
273     'Bored': {
274         'emoji': '😴',
275         'response': 'Let's make this more engaging. Here are some interactive exercises...',
276         'action': 'Show interactive content'
277     },
278     'Curious': {
279         'emoji': '🤔',
280         'response': 'Excellent question! Here's more in-depth information...',
281         'action': 'Provide research papers & advanced materials'
282     }
283 }
```

Response Template Fallback

Story 3: Response Regeneration and Synchronization Mechanism

Description

- Provides user control to toggle AI response generation on/off. When unchecked, system falls back to template responses. Allows users to switch between AI-generated and template responses dynamically.

```
app.py 5 X
app.py > ...

184 def main():
248
249     with col2:
250         st.subheader("⚙️ Settings")
251         use_ai = st.checkbox("Use AI Response (Gemini)", value=True)
252         save_data = st.checkbox("Save to CSV for learning", value=True)
253         show_details = st.checkbox("Show analysis details", value=False)
```

AI Toggle Control

- Regenerates responses each time the "Get AI Learning Help" button is clicked, using current input text and AI toggle setting. Keeps emotion scores synchronized with response generation by using the same prediction results for both display and response creation.

Response Regeneration Logic

- Displays emotion scores and predictions in real-time, ensuring that response generation uses the same emotion and confidence values shown in the UI. Maintains consistency between displayed predictions and AI response generation.

Story 4: Session History Management and CSV Logging

Description

- Initializes session history list in Streamlit session state and adds each interaction with timestamp, field, problem, emotion (including mixed emotions), confidence, AI response, all emotion scores, and model type. Maintains separate entries for BiLSTM and BERT predictions when both are available.

```
def add_to_history(field, problem, emotion, confidence, ai_response, bilstm_scores, bert_result=None):
    # Detect mixed emotions for history
    def get_mixed_emotions(scores, threshold=0.15):
        sorted_emotions = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        primary = sorted_emotions[0]
        mixed = [primary]

        for emotion_name, score in sorted_emotions[1:]:
            if score >= threshold:
                mixed.append((emotion_name, score))

        return mixed if len(mixed) > 1 else [primary]

    mixed_emotions = get_mixed_emotions(bilstm_scores)
    emotion_label = " + ".join([em[0] for em in mixed_emotions]) if len(mixed_emotions) > 1 else emotion

    # Add BiLSTM entry
    st.session_state.emotion_history.append({
        'timestamp': datetime.now(),
        'field': field,
        'problem': problem,
        'emotion': emotion_label,
        'confidence': confidence,
        'ai_response': ai_response,
        'all_scores': bilstm_scores,
        'model': 'BiLSTM'
    })

    # Add BERT entry if available
    if bert_result:
        bert_mixed = get_mixed_emotions(bert_result['scores'])
        bert_emotion_label = " + ".join([em[0] for em in bert_mixed]) if len(bert_mixed) > 1 else bert_result['emotion']

        st.session_state.emotion_history.append({
            'timestamp': datetime.now(),
            'field': field,
            'problem': problem,
            'emotion': bert_emotion_label,
            'confidence': bert_result['confidence'],
            'ai_response': ai_response,
            'all_scores': bert_result['scores'],
            'model': 'BERT'
        })
```

Session History Management

- Saves each interaction to `emotion_response_examples.csv` with text, emotion, confidence, response, field, and ISO-formatted timestamp. Updates `emotion_response_mapping.csv` with new emotion-response pairs for future reference. Handles file creation and appending gracefully.

CSV Logging Implementation

- Displays session statistics in sidebar including total interactions count, CSV

```
def save_to_csv(field, problem, emotion, confidence, ai_response):
    """Save new interaction to CSV files."""
    try:
        # Update examples CSV
        new_example = {
            'text': problem,
            'emotion': emotion.lower(),
            'confidence': confidence,
            'response': ai_response,
            'field': field,
            'timestamp': datetime.now().isoformat()
        }

        # Check if file exists
        if os.path.exists("emotion_response_examples.csv"):
            df = pd.read_csv("emotion_response_examples.csv")
            df = pd.concat([df, pd.DataFrame([new_example])], ignore_index=True)
        else:
            df = pd.DataFrame([new_example])

        df.to_csv("emotion_response_examples.csv", index=False)

        # Update mapping CSV if new emotion-response pair
        if os.path.exists("emotion_response_mapping.csv"):
            mapping_df = pd.read_csv("emotion_response_mapping.csv")
            if emotion not in mapping_df['emotion'].values:
                new_mapping = pd.DataFrame([{'emotion': emotion, 'response': ai_response}])
                mapping_df = pd.concat([mapping_df, new_mapping], ignore_index=True)
            mapping_df.to_csv("emotion_response_mapping.csv", index=False)

        return True
    except Exception as e:
        st.error(f"Failed to save to CSV: {e}")
        return False
```

examples count, and recent session summaries. Provides "Clear History" button to reset session state. Shows last 3 interactions with field, emotion, and confidence for quick reference.

```
# Sidebar
with st.sidebar:
    st.header("📊 Dashboard")
    st.write(f"Models: {status}")
    st.write(f"Total Interactions: {len(st.session_state.emotion_history)}")
    st.write(f"CSV Examples: {len(examples_df)}")

    if st.button("Clear History"):
        st.session_state.emotion_history = []
        st.rerun()

# Recent Interactions
if st.session_state.emotion_history:
    st.subheader("Recent Sessions")
    recent = st.session_state.emotion_history[-3:]
    for item in reversed(recent):
        st.write(f"* {item['field']}: {item['emotion']} ({item['confidence']:.1%})")
```

Session History Display

Epic 5. Streamlit UI Implementation

Story 1: Responsive Layout and Session State Management Description

Configures Streamlit page with wide layout for better use of screen space. Initializes session state dictionary to store emotion history, maintaining data persistence across user interactions within the same session.

```
st.set_page_config(page_title="AI Learning Assistant", page_icon="🤖", layout="wide")

# Initialize session state
if 'emotion_history' not in st.session_state:
    st.session_state.emotion_history = []
```

Page Configuration & Session State Initialization

- Implements sidebar dashboard displaying model status, interaction counts, and recent session summaries. Provides "Clear History" button to reset session state. Shows last 3 interactions with field, emotion, and confidence for quick reference.

```
# Sidebar
with st.sidebar:
    st.header("📊 Dashboard")
    st.write(f"Models: {status}")
    st.write(f"Total Interactions: {len(st.session_state.emotion_history)}")
    st.write(f"CSV Examples: {len(examples_df)}")

    if st.button("Clear History"):
        st.session_state.emotion_history = []
        st.rerun()

# Recent interactions
if st.session_state.emotion_history:
    st.subheader("Recent Sessions")
    recent = st.session_state.emotion_history[-3:]
    for item in reversed(recent):
        st.write(f"• {item['field']}: {item['emotion']} ({item['confidence']:.1%})")
```

Sidebar Dashboard

- Creates dedicated section for field selection (11 academic subjects) and problem description input. Provides contextual placeholders based on selected field to guide user input. Uses text area with fixed height for consistent layout.

```
with col1:
    st.subheader("🗨️ Tell us about your learning challenge")

    # Field selection
    field = st.selectbox(
        "What field are you studying?",
        ["Computer Science", "Mathematics", "Physics", "Chemistry", "Biology",
         "Engineering", "Business", "Literature", "History", "Psychology", "Other"],
        help="Select your area of study for personalized responses"
    )

    # Problem description
    problem = st.text_area(
        f"Describe your {field} problem or challenge:",
        placeholder=f"e.g., 'I'm struggling with algorithms in {field}' or 'This concept is confusing'",
        height=120
    )
```

Sidebar Dashboard

Story 2: Emotion Analysis, Model Comparison, and Visualization Components

Description

```
with col2:
    st.subheader("⚙️ Settings")
    use_ai = st.checkbox("Use AI Response (Gemini)", value=True)
    save_data = st.checkbox("Save to CSV for learning", value=True)
    show_details = st.checkbox("Show analysis details", value=False)

    # CSV Prediction Option
    st.markdown("---")
    st.write("**📄 Predict from Saved Data**")
    use_csv_prediction = st.checkbox("Use CSV-based prediction", value=False)

    if use_csv_prediction and len(examples_df) > 0:
        st.info(f"Using {len(examples_df)} saved examples for prediction")
```

```
for emotion_name, score in sorted(bilstm_result['scores'].items(), key=lambda x: x[1], reverse=True):
    st.progress(score, text=f"{emotion_name}: {score:.1%}")

if bert_result:
    with col2:
        st.write("***BERT Transformer**")
        bert_mixed = get_mixed_emotions(bert_result['scores'])

        if len(bert_mixed) > 1:
            mixed_text = " + ".join([f"{EMOTION_RESPONSES[em[0]]['emoji']} {em[0]}" for em in bert_mixed])
            st.metric("Mixed Emotions", mixed_text, f"Primary: {bert_mixed[0][1]:.1%}")
        else:
            bert_emoji = EMOTION_RESPONSES[bert_result['emotion']]['emoji']
            st.metric("Emotion", f"{bert_emoji} {bert_result['emotion']}", f"{bert_result['confidence']:.1%}")

    for emotion_name, score in sorted(bert_result['scores'].items(), key=lambda x: x[1], reverse=True):
        st.progress(score, text=f"{emotion_name}: {score:.1%}")
```

- Displays side-by-side comparison of BiLSTM and BERT model predictions when both are available. Shows mixed emotions with emojis when multiple emotions are detected, or single emotion with confidence percentage. Displays progress bars for all emotion scores in descending order.

Model Comparison Display (BiLSTM vs BERT)

Story 3: Form Controls, Validation, and Error Handling

Description

```
# Analytics Dashboard
if st.session_state.emotion_history:
    st.markdown("---")
    st.header("📊 Learning Analytics")

    df = pd.DataFrame(st.session_state.emotion_history)

    tab1, tab2, tab3 = st.tabs(["Emotions", "Fields", "Summary"])
```

- Creates tabbed interface for analytics dashboard with three tabs: "Emotions" (emotion distribution and timeline), "Fields" (field-based breakdowns), and "Summary" (overall statistics). Only displays when session history exists.

Analytics Tabs Structure

- Provides settings panel with checkboxes for AI response toggle, CSV saving, analysis details display, and CSV-based prediction option. Shows informative messages when CSV prediction is enabled with available examples.

Settings Panel & Form Controls

Story 4: Analytics Dashboard and Interactive Charts

Description

- Creates interactive pie chart showing emotion distribution across all sessions. Uses Plotly Express for interactive visualization with hover tooltips. Displays in first column of "Emotions" tab with responsive width.

```
with tab1:
    col1, col2 = st.columns(2)
    with col1:
        emotion_counts = df['emotion'].value_counts()
        fig1 = px.pie(values=emotion_counts.values, names=emotion_counts.index,
                      title="Emotion Distribution")
        st.plotly_chart(fig1, use_container_width=True)
```

Emotion Distribution Pie Chart

- Generates interactive line chart showing confidence timeline across sessions. Color-codes lines by emotion type with markers for each data point. Displays "Emotional Journey" showing how student confidence and emotions change over time.

```
with col2:
    df_copy = df.copy()
    df_copy['time'] = df_copy['timestamp'].dt.strftime('%H:%M:%S')
    fig2 = px.line(df_copy, x='time', y='confidence', color='emotion',
                  title="Emotional Journey", markers=True)
    st.plotly_chart(fig2, use_container_width=True)
```

Confidence Timeline Chart

- Creates interactive bar chart showing emotion distribution by study field. Supports faceting by model type when both BiLSTM and BERT predictions are available. Color-codes bars by emotion for easy comparison across fields.

```
with tab2:
    if 'model' in df.columns:
        field_emotion = df.groupby(['field', 'emotion', 'model']).size().reset_index(name='count')
        fig3 = px.bar(field_emotion, x='field', y='count', color='emotion', facet_col='model',
                      title="Emotions by Study Field & Model")
    else:
        field_emotion = df.groupby(['field', 'emotion']).size().reset_index(name='count')
        fig3 = px.bar(field_emotion, x='field', y='count', color='emotion',
                      title="Emotions by Study Field")
    st.plotly_chart(fig3, use_container_width=True)
```

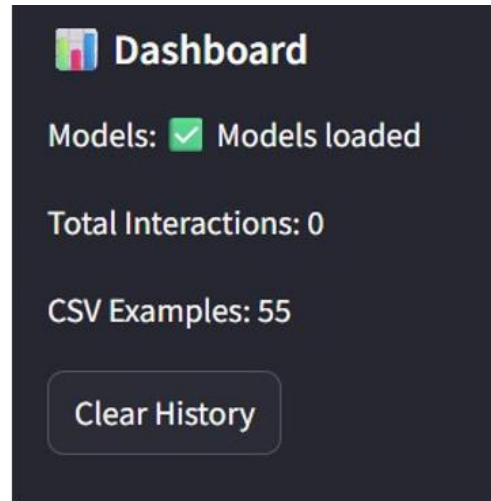
Confidence Timeline Chart

Epic 6. User Interaction

Story 1: Validate UI Flow End-to-End

Description

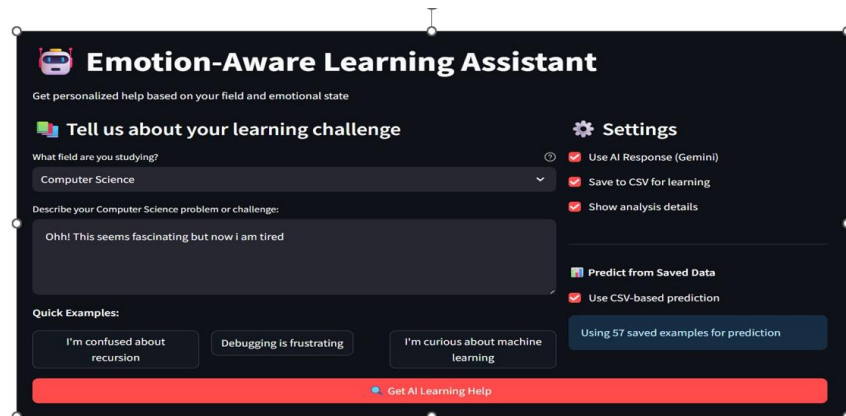
- Tests the complete user journey from input to response, verifying all UI components, navigation, and user interactions work correctly. Ensures smooth user experience across



all features.

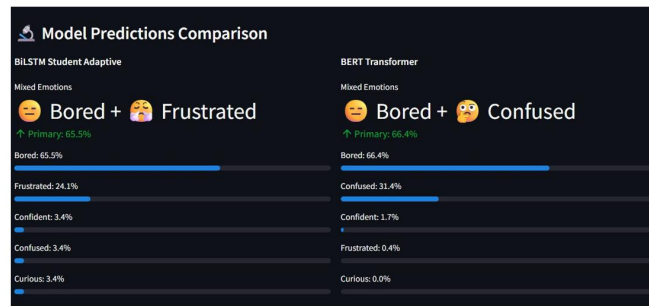
Application Startup & Model Loading

Validates that models load successfully on application startup. Confirm BiLSTM and BERT models are cached and ready for inference. Shows initial state with no interactions yet.

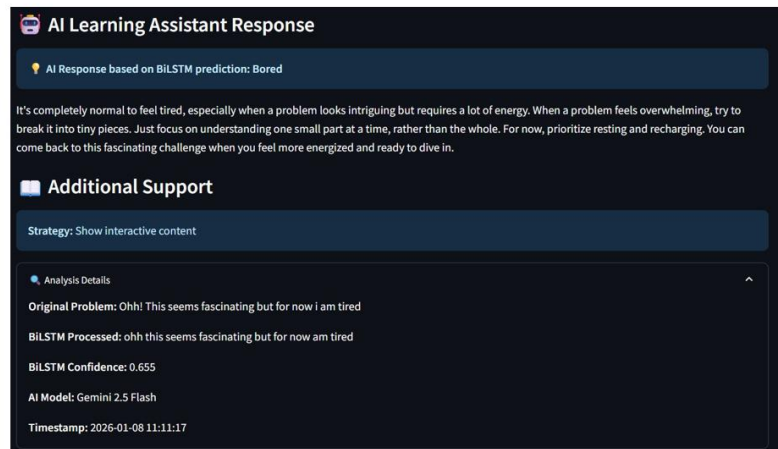


Input Form Validation

- Validates end-to-end flow from button click through model inference to response display. Confirms all UI sections update correctly with prediction results.
- Verifies mixed emotion detection works correctly, displaying multiple emotions when confidence scores exceed threshold. Confirms visual representation matches calculated results.

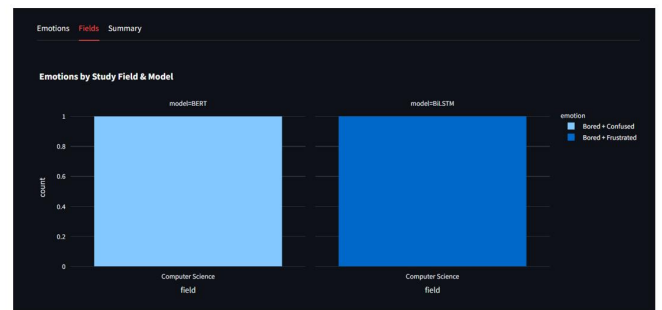
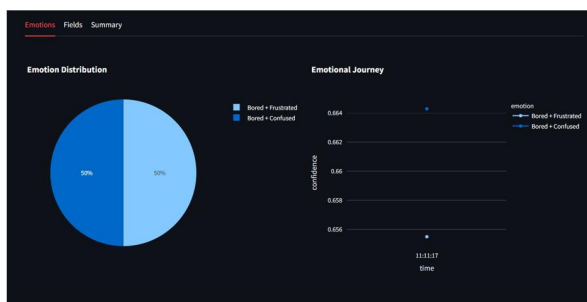


Model Predictions Comparison section



AI Response section with generated text

- Validates analytics dashboard navigation and tab functionality.
- Confirms all three tabs (Emotions, Fields, Summary) render correctly with appropriate visualizations



Analytics Dashboard Navigation

Story 2: Optimization and Deployment Readiness

Description

- **Cross-Browser Verification**

- Tested application in multiple browsers.
- Confirmed consistent layout, widgets, and visualizations.
- Verified absence of browser-specific UI errors

- **Performance and Caching**

- Confirmed models are loaded only once.
- Verified caching of stored data for faster responses.
- Observed stable response times across repeated interactions.

- **Environment and Final Validation**

- Verify environment variables are correctly configured.
- Confirmed required models and dependencies are available.
- Validated complete end-to-end workflow functionality.
- Confirmed system readiness for deployment.

- **Slide Splitting & PNG Conversion Testing**

Validate that each slide correctly produces:

- one PPTX file
- one PNG preview

Conclusion:

The AI-Powered Learning Assistant represents a significant advancement in how students and educators understand and respond to emotional states during learning. By combining robust text preprocessing, a BiLSTM student-adaptive model, a fine-tuned BERT classifier, and

mixed-emotion detection, the system can interpret nuanced learner emotions across five key categories with high accuracy and transparency. On top of this emotion layer, Gemini-driven

response generation and carefully designed fallback templates deliver empathetic, field-aware

guidance that helps students feel understood while receiving concrete, actionable support.

Built on a modular architecture with reusable Kaggle training notebooks, exported models, and a Streamlit-based interface, the platform offers both technical robustness and strong usability. The dashboard, analytics tabs, and CSV logging enable continuous learning and rich insights into emotional patterns across sessions and subjects, making it valuable for individual learners, tutors, and institutions alike. Looking forward, the assistant provides a strong foundation for extensions such as multilingual emotion detection, voice or multimodal inputs, deeper LMS integration, adaptive learning paths, and instructor dashboards. With its blend of neural models, rule-based enhancements, and human-centered design, the AI Learning Assistant sets a practical benchmark for next-generation emotion-aware educational technology.

